

P2490R0

Zero-overhead exception stacktraces

Published Proposal, 2021-11-13

This version:

<https://wg21.link/P2490>

Issue Tracking:

[GitHub](#)

Author:

[Ed Catmur](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ecatmur/stacktrace-from-exception/blob/main/paper.md

Table of Contents

1 Overview

1.1 Abstract

1.2 Background

1.2.1 Runtime cost

1.2.2 Old third-party libraries

1.2.3 Conflicting requirements

1.3 Alternatives

1.4 Acknowledgements

1.5 Revision history

2 Possible syntaxes

2.1 Special function

- 2.2 Default parameter
- 2.3 "catch-with-init"
- 2.4 Expose search phase

3 Concerns

4 Implementation experience

- 4.1 Windows
- 4.2 Itanium

References

Informative References

§ 1. Overview

§ 1.1. Abstract

This paper identifies concerns with part of the **Stacktrace from exception** [p2370] proposal. We suggest alternate approaches and offer implementation experience of the techniques that could underly such alternatives.

§ 1.2. Background

The paper **Stacktrace from exception** [p2370] amply sets out why it is desired to be able to access a stacktrace from exception; that is, when *handling* an exception it should be possible to retrieve a stacktrace from the (most recent) **throw** point of the exception, through the point of handling; and that this should be *transparent* to and not require *cooperation by* or *modification of* throwing code. That paper acknowledges that the cost of taking a stacktrace on *every* exception throw would be prohibitive and proposes a mechanism to disable it via a standard library routine `std::this_thread::set_capture_stacktraces_at_throw` that will set a thread-local flag.

We argue that this mechanism still imposes a runtime cost and would not achieve the aims of the paper for the proposed facility.

§ 1.2.1. Runtime cost

Accessing a thread-local variable has a cost in instructions and memory access; at present this could be argued to be lost in the "noise" of the existing exception handling machinery, particularly as this currently involves memory allocation, but in future if and when the **Zero-overhead deterministic exceptions: Throwing values** [\[p0709\]](#) proposal is adopted this will become relatively more significant.

In any case even a *de minimis* runtime cost is not zero.

§ 1.2.2. Old third-party libraries

Under the proposed mechanism, throw sites would need recompilation and/or relinking to participate in the facility. It is entirely possible that third-party library code is shipped with its own implementations of the exception-raising mechanism, such that it would not participate in the facility until such time as the vendor recompiles and relinks the library, which may not come for years or even decades.

§ 1.2.3. Conflicting requirements

Third-party library vendors who use exceptions for control flow may be expected to view the proposed facility negatively; if user code enables it via the proposed mechanism the cost will be considerable even for exceptions that are caught and handled successfully entirely within the third-party library. Thus they are likely to disable the facility at API entry points, both negating the point of the facility for any exceptions that *do* leak out of the third-party library, and interfering with user code that expects it to remain enabled.

§ 1.3. Alternatives

We note that C++ exception handling is typically built on top of a lower-level, language-agnostic facility. On Windows this is structured exception handling [\[seh\]](#), while on the Itanium ABI (used by most Unix-style OSes on x64-64) it is the Level I Base ABI [\[itanium\]](#). This lower-level facility uses *two-phase* exception handling; in the first, "search" phase the stack is walked from the throw point to identify a suitable handler, while in the second, "unwind" phase it is walked again from the throw point to the selected handler, this time invoking cleanup (i.e., destructors) along the way. Importantly,

- during the whole of the search phase the stack is still intact, and

- identifying a handler is *dynamic*, calling into a compiler- or library-generated match function.

This suggests a possible alternative mechanism; viz.:

- user code can mark a `catch` block as requiring a stacktrace, either via a special function or via new syntax; and
- on recognising `catch` blocks so marked, the compiler can emit suitable code or data such that *if and when* that `catch` block is selected as a handler for a thrown exception, it takes a stacktrace during search phase, immediately before nominating that `catch` block as a suitable handler; then
- the user code can retrieve that stored stacktrace during exception handling, after stack unwinding.

The advantages of this approach are twofold:

- *transparency*: there is no need whatsoever for throwing code to be modified, recompiled or relinked. Indeed, since this mechanism relies solely on changes to the catch site, code using this mechanism may be introduced into existing (perhaps even running) programs without any need for those programs to be recompiled or relinked, as long as that new code has access to any necessary support libraries;
- *zero-cost*: if the search phase does not reach a `catch` block so marked (i.e., if the exception is caught and handled internally) then behavior is entirely unaffected.

§ 1.4. Acknowledgements

Thank you especially to Antony Peacock for getting this paper ready for initial submission, and to Mathias Gaunard for inspiration, review and feedback.

§ 1.5. Revision history

R0

Initial revision; incorporated informal feedback.

§ 2. Possible syntaxes

We now propose a number of possible syntaxes for user code to opt in to the mechanism and retrieve the stored stacktrace.

§ 2.1. Special function

Under this mechanism, a `catch` block that contains a potentially-evaluated call to `std::stacktrace::from_current_exception()` is marked as requiring a stacktrace to be taken during search phase:

```
try {  
    ...  
} catch(std::exception& ex) {  
    std::cout << ex.what() << "\n" << std::stacktrace::from_current_excepti  
}
```

Drawback: if the call is moved out of the `catch` block, e.g. to a helper function, or even perhaps to a lambda within it, the facility will cease to work. Possible workaround: make `std::stacktrace::from_current_exception()` ill-formed when called from anywhere other than a `catch` block - this could be ugly.

§ 2.2. Default parameter

```
try {  
    ...  
} catch(std::exception& ex, std::stacktrace st = std::stacktrace::from_curr  
    std::cout << ex.what() << "\n" << st << std::endl;  
}
```

This can be understood by analogy to `std::source_location::current()` which similarly has special behavior in particular contexts.

Drawbacks: new syntax, could be understood as providing multiple types to be caught.

§ 2.3. "catch-with-init"

```
try {  
    ...  
} catch(auto st = std::stacktrace::current(); std::exception& ex) {
```

```
std::cout << ex.what() << "\n" << st << std::endl;
}
```

Here we add an (optional) *init-statement* to the `catch` clause, to be executed before unwind (if the catch clause is selected).

Drawbacks: new syntax, may not be safe to run general user code during search phase.

§ 2.4. Expose search phase

```
try {
    ...
} catch(std::exception& ex) if (auto st = std::stacktrace::current(); true)
    std::cout << ex.what() << "\n" << st << std::endl;
}
```

Drawbacks: new syntax, open to abuse, may not be safe to run general user code during search phase.

§ 3. Concerns

We do not know whether this mechanism is indeed implementible on all platforms. We do know that (and, indeed, have practical experience [§ 4 Implementation experience](#) to show that) it is implementable on two major platforms (i.e. Windows on Intel, and Unix-like on x86-64) that between them cover a dominant proportion of the market. We would welcome information regarding alternative platforms.

Third-party vendors who view secrecy as a virtue may be tempted to put `catch ( ... )` blocks at API entry points to prevent information on their library internals leaking out. In practice they can achieve much the same end by stripping debug symbols and obfuscating object names, and are likely to do so; meanwhile the same information is available by attaching a debugger.

Some of the proposed mechanisms [§ 2 Possible syntaxes](#) are potentially confusing or open to abuse.

For a *rethrown* exception (using `throw`; or `std::rethrow_exception`) the stacktrace will only extend as far as the rethrow point. We could provide mechanisms to alleviate this, either opt-in or opt-out; for example, adding `std::current_exception_with_stacktrace` or marking `catch` blocks containing `throw`; as requiring stacktrace.

§ 4. Implementation experience

The following implementations are provided solely to demonstrate implementability; we anticipate that any Standard implementation would be significantly less ugly in both internals and in use.

§ 4.1. Windows

It is well known that the vendor-specific `__try` and `__except` keywords [try-exception] (present in Visual Studio and compatible compilers) permit arbitrary code to be invoked during search phase, since the argument to the `__except` keyword is a *filter-expression* evaluated during search phase, to an enumeration indicating whether the consequent code block is to be selected as the handler. We present a proof-of-concept implementation [poc] (32-bit and 64-bit) adapted from an article by Howard Jeng [jeng].

§ 4.2. Itanium

Although exception handling on Itanium is also two-phase, the handler selection mechanism is largely hidden from the user. However, there is a workaround involving creating a type whose run-time type information (that is, its `typeid`) refers to an instance of a user-defined subclass of `std::type_info`. This technique is not particularly widely known, but has been used in several large proprietary code bases to good effect for some time. We present a proof-of-concept implementation [poc].

§ References

§ Informative References

[ITANIUM]

C++ ABI for Itanium: Exception Handling. URL: <https://itanium-cxx-abi.github.io/cxx-abi/abi-eh.html>

[JENG]

The Visual C++ Exception Model. URL: <https://www.gamedev.net/tutorials/programming/general-and-gameplay-programming/the-visual-c-exception-model-r2488/>

[P0709]

Zero-overhead deterministic exceptions: Throwing values. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0709r4.pdf>

[P2370]

Stacktrace from exception. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2370r1.html>

[POC]

Proof-of-concept implementation.. URL: <https://github.com/ecatmur/stacktrace-from-exception/blob/main/stacktrace-from-exception.cpp>

[SEH]

Structured Exception Handling (C/C++). URL: <https://docs.microsoft.com/en-us/cpp/cpp/structured-exception-handling-c-cpp>

[TRY-EXCEPT]

`try-except` statement. URL: <https://docs.microsoft.com/en-us/cpp/cpp/try-except-statement>